

Synthesizing an Optimal Spin Qubit Shuttling Bus Architecture for the Surface Code

Pau Escofet,^{*} Eduard Alarcón, and Sergi Abadal
*Universitat Politècnica de Catalunya,
 Barcelona, Spain*

Andrii Semenov, Niall Murphy, and Elena Blokhina[†]
*Equal1 Labs,
 Dublin, Ireland*

Carmen G. Almudéver
*Universitat Politècnica de València,
 València, Spain*

As quantum computers scale toward millions of physical qubits, it becomes essential to robustly encode individual logical qubits to ensure fault tolerance under realistic noise. A high-quality foundational encoding allows future compilation techniques and heuristics to build on optimal or near-optimal layouts, improving scalability and error resilience. In this work, we synthesize a one-dimensional shuttling bus architecture for the rotated surface code, leveraging coherent spin-qubit shuttling, following a novel methodology we name *Quantum Reverse Mapping*. We formulate a mixed-integer optimization model that yields optimal solutions with relatively low execution time for small code distances, and propose a scalable heuristic that matches optimal results while maintaining linear computational complexity. We evaluate the synthesized architecture using architectural metrics, such as shuttling distance and cycle time, and full quantum simulations under realistic noise models, showing that the proposed design can sustain logical error rates as low as 2×10^{-10} per round at code distance 21, showcasing its feasibility for scalable quantum error correction in spin-based quantum processors.

I. INTRODUCTION

Quantum error correction (QEC) is widely recognized as a requisite for scaling quantum processors to the fault-tolerant regime [1–3]. By detecting and correcting errors in quantum states, QEC decreases the error rates and makes the quantum states robust for the large-scale application of quantum algorithms [4, 5].

Among the many families of QEC codes, topological surface codes stand out for their high threshold, reliance on local nearest-neighbor interactions, and the existence of efficient decoding algorithms [6–11].

The performance of an error-corrected logical qubit depends on multiple factors, including the chosen error-correction code, the underlying noise processes, and the available architectural primitives and connectivity. Much of the current state-of-the-art research focuses on how to encode a logical qubit as robustly as possible given a particular code and a fixed physical architecture. For instance, the surface code requires only nearest-neighbor interactions, which map naturally onto superconducting architectures with two-dimensional grid connectivity [9, 11–13]. The surface code has also been adapted to alternative qubit technologies, requiring a dedicated mapping layer to reconcile the code’s 2D connectivity with the physical architecture [14–16].

In this work, we take the opposite approach. Rather than starting from a fixed hardware architecture and determining how best to embed a quantum error correction code into it, we synthesize a spin-qubit-based architecture that is designed from the ground up to encode a single logical qubit as efficiently and robustly as possible under realistic architectural constraints. We name this methodology **Quantum Reverse Mapping**, as it inverts the traditional perspective: rather than adapting the code to a fixed hardware platform, we derive the hardware layout directly from the intrinsic requirements of the error-correcting code, effectively reverse-synthesizing an architecture that optimally supports it.

Looking forward to large-scale quantum processors, the problem shifts from the mapping of a single logical, error-corrected qubit to the compilation of full quantum circuits consisting of many logical qubits. At that scale, optimally compiling logical circuits onto physical layouts is intractable, as the underlying mapping and routing problems are NP-complete [17]. As a result, practical compilers rely on heuristic routing strategies and scheduling algorithms that provide good but inherently sub-optimal solutions within reasonable time [18–20]. The effectiveness of such heuristics, however, depends critically on the quality of the logical-qubit encoding at the lowest layer. By establishing an optimized encoding of a single logical qubit, we provide a strong foundation that future heuristic compilation layers can build upon, thereby improving scalability and noise resilience.

On a lower layer, silicon spin qubits [21–23] have

^{*} pau.escofet@upc.edu

[†] Also at University College Dublin

emerged as an especially attractive hardware platform for industrially scalable devices; they combine long coherence times [24], gate fidelities exceeding 99% [25–27], and compatibility with current semiconductor manufacturing processes, allowing for easier integration with classical control electronics [28].

Another interesting property of spin qubits is their native capability for qubit shuttling [29–32]. By transporting qubits along 1D channels (often called shuttling tracks or buses) we can dynamically bring qubits into proximity precisely when interactions are required. This mechanism effectively underlies many promising, scalable architectures for semiconductor spin-qubit platforms [33–36].

Shuttling operations, as we discuss below, are fundamental to the functionality of future spin-based quantum processors. Due to practical limitations, long-range dot-to-dot coupling across extended quantum devices is not feasible for large-scale architectures. Instead, quantum states must be transferred between distant sites through reliable and coherent state transfer mechanisms.

Qubit shuttling emerged as one of the most promising mechanisms for silicon spin qubits. It offers a potential pathway to mitigate connectivity constraints. By physically moving an electron or hole within a quantum dot array or tile, shuttling enables flexible routing of quantum information, access to charge sensors and effective long-range connectivity. Across some notable works on shuttling architectures for spin qubits, we note the following architectures. Study [37] proposes one of the earliest architectures for quantum computation using semiconductor spins qubits. The architecture proposes to use two adjacent gate-defined double quantum dots, high-frequency microwave gates and capacitive coupling gates to form a node. Alternating gates are used to shuttle the spin, and qubits are connected by a spin shuttle track (“electron pump”). The study proposes a universal set of quantum gates built for singlet-triplet qubits with charge coupling, which facilitates active error suppression, and outlines a scalable architecture with favourable error thresholds for fault-tolerant operation.

Study [38] proposes a 2D donor electron-spin quantum computer architecture enabled by an electron spin transport mechanism. Shuttling of spins enables 2D scaling and is a core architectural feature of the study. Paper [36] introduces SpinBus, an architecture that explicitly uses electron shuttling in Si/SiGe to connect qubits as a scaling strategy. This architecture emphasises that spin shuttling requires low operating frequencies for maintaining qubit coherence.

Another recent discussion on shuttling architecture is presented in [39]. Spin shuttling is used to move qubits into an interaction zone. Reference [40] addresses a Si/SiGe-specific issue – valley excitations near valley-splitting minima during shuttling. This study proposes a 2D shuttling pattern that enables omnidirectional motion for network routing.

Compilation ideas are presented in [41]. This study

considers a sparse 2D connectivity and proposes a modular 2D spin-qubit architecture built from silicon spin qubits. The paper introduces a silicon “junction” that can couple up to four nanowires into 2D arrangements using spin shuttling and SWAP operations.

Two critical parameters define the performance of any shuttling system: the shuttling duration, which must be significantly shorter than the qubit coherence time, and the amount of additional decoherence introduced during the transfer. In the literature, two primary shuttling modes have been established: bucket-brigade and conveyor-mode. In the bucket-brigade mode [29, 32], a carrier (an electron or a hole) is transferred from dot to dot via tunnelling, which leads to increased sensitivity to charge noise. In contrast, conveyor-mode shuttling [31, 42] transports the carrier by smoothly translating the confining potential well, thereby eliminating tunnelling events and associated noise sources [43], making it a promising candidate for scalable implementations.

Experimentally, conveyor-mode shuttling is typically implemented by applying phase-shifted, periodic voltages to a sequence of control electrodes [43, 44]. This creates a moving potential landscape that carries the quantum state across the device. Sinusoidal waveforms are often employed due to their simplicity; they are not necessarily optimal for minimizing decoherence or maximizing shuttling fidelity, and more sophisticated control strategies may offer improved performance [45, 46].

While shuttling provides increased architectural flexibility, different material platforms present distinct advantages and challenges. In Si/SiGe heterostructures, conveyor-mode electron shuttling is generally considered less noisy than bucket-brigade shuttling; however, variations in valley splitting [47], material disorder, potential roughness, wiring density, and heating effects [48] can introduce additional sources of decoherence. In germanium-based structures, hole shuttling benefits from strong spin-orbit coupling but may face different noise mechanisms. In GaAs-based structures, the absence of valley degeneracy allows for potentially smoother shuttling, but the presence of non-zero nuclear spins leads to strong hyperfine interactions, significantly reducing the spin coherence time T_2^* compared to Si/SiGe heterostructures [49, 50]. In Si/SiGe heterostructures, an isotopically purified Si-28 channel is used to enhance the coherence of the spin, but the small valley splitting energy might be taken into account through different architecture solutions as we discussed above [40, 51].

In this work we synthesize an optimal one-dimensional shuttling-bus architecture [35, 36] for the rotated surface code. Our approach combines a mixed-integer optimization model for small code distances with a scalable heuristic for larger distances, by exploiting coherent spin shuttling [31, 32]. The resulting architecture minimizes both the total shuttling distance and cycle latency, obtaining improved logical error rates compared to the baseline architecture synthesis method.

Established layout optimization techniques (including

classical standard VLSI placement [52, 53] and emerging quantum computing approaches [54]) are inapplicable to the unique properties and constraints of the considered architecture, which leverages 1D global parallel shuttling. Consequently, our synthesis techniques are specifically tailored to the architecture at hand, aligning with the growing paradigm of application-aware co-design of quantum systems, where specific architectures are tailored to algorithmic requirements to maximize efficiency under hardware constraints [55–58].

The main contributions of this work are as follows:

- The development of a mathematical model for optimally synthesizing spin qubits in a shuttle bus architecture, minimizing the error overhead added by the shuttlings needed for the syndrome extraction of the chosen quantum error correction code.
- The proposal of an optimal-inspired heuristic placement technique for the scalable synthesis of larger code distances specifically tailored for the surface code.
- An analysis of the pseudo-threshold values for each noise source of the considered noise model, consisting of depolarizing noise for quantum gates and dephasing noise for latency-induced errors.
- An exploration of the architecture design decisions’ impact on the performance of the surface code, considering shuttling velocity, distance between elements, and architecture synthesis techniques.

The remainder of the paper is structured as follows. In Section II we introduce the considered architecture and review similar work. Section III explains how the surface code can be mapped into the architecture and explores optimal and heuristic qubit placement. In Section IV we explore how the proposed layout performs for the surface code, exploring both noise and architecture parameters. The work is concluded in Section V where the outcomes of the paper are summarized and future work directions are outlined.

II. THE SHUTTLING BUS ARCHITECTURE

Large-scale spin-qubit processors face a severe pin-count bottleneck: each physical qubit typically demands multiple DC, pulsed, or microwave control lines, so that the required number of classical wires grows with the number of qubits, whereas the available routing cross-section grows only with the perimeter of the processor [59]. Architectures such as cross-bar [60] and bilinear layouts [61] alleviate, but do not eliminate, the fan-out problem.

A promising remedy is to arrange qubits along a 1-D “conveyor” track and shuttling them to fixed gate regions only when interactions are required. Patomäki *et al.*

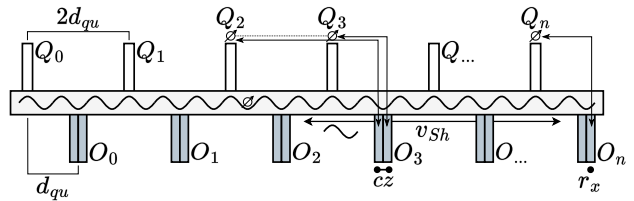


FIG. 1: Sketch of the proposed shuttling bus architecture. Spins in quantum dots $\{Q_0, \dots, Q_n\}$ are shuttled to manipulation zones $\{O_0, \dots, O_n\}$ when they have to interact. The distance between consecutive elements in the 1D shuttling bus is d_{qu} , and the shuttling velocity of the spins is set to v_{sh} .

introduced a pipeline architecture in which qubits advance column-by-column through predefined single- and two-qubit gate sites under global timing signals [35]. Künne *et al.* proposed the SpinBus architecture, a modular array of parallel shuttling lanes that provides 2D logical connectivity while requiring only four phase-shifted sinusoidal voltages per lane [36]. Both schemes leverage recent demonstrations of coherent electron-spin shuttling [31, 32].

In this work, we adopt a single-lane variant of these proposals, as depicted in Figure 1. We call this architecture the *shuttling bus architecture*, which is divided into storage and manipulation zones. Storage zones are quantum dots where qubits reside idly, these storage zones being connected through a shuttling bus mechanism to manipulation zones, which are designated areas for single- and two-qubit gates, as well as readout operations.

A global four-phase waveform translates the confinement minimum along the track, conveying qubits from storage zones to manipulation zones each time they must interact. Because the waveform is broadcasted, the number of cryogenic control signals is independent of the number of qubits, directly addressing the perimeter-vs-area imbalance posed before [31, 36].

Qubits can be in storage or manipulation zones. They are moved from one zone to another via the shuttling bus connecting all the zones. This means that, if two qubits are on a track they should move in the same direction (either right or left), and at the same speed. Quantum gates can only be applied to qubits located in manipulation zones. Therefore, each time a gate needs to be executed, the associated qubit must be shuttled from a storage zone to a manipulation zone, where the gate will be applied. Moreover, for the execution of two-qubit gates, the two qubits involved must be in the same manipulation zone.

Previous studies have examined similar systems. In [62] a shuttle based looped-pipeline architecture is used to emulate 3D connectivity within a strictly 2D device for the surface code. More recently, [63] also studied the surface code implementation in the SpinBus architecture. In parallel, other works focused on proposing compilation techniques for circuit execution in shuttling-

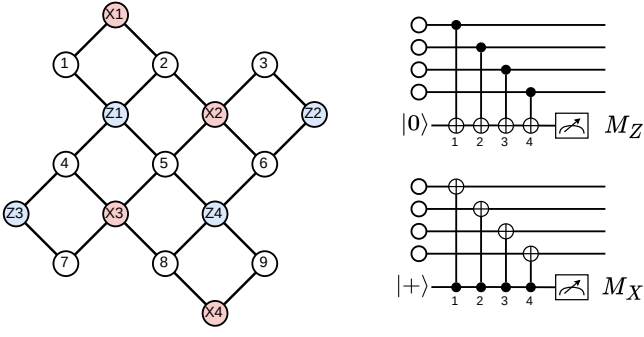


FIG. 2: The 2D planar layout of the distance-3 surface code (left). Data qubits are represented with white nodes and X and Z checks are red and blue nodes respectively. Top and bottom circuits (right) are Z - and X -syndrome extraction circuits, respectively, each consisting on four CNOT gates.

based architectures [41, 64, 65].

In the following sections, we used the proposed architecture to store a single logical qubit using the surface code, assessing how different layouts (*i.e.*, different distributions of storage and manipulation zones) perform, using both architectural metrics and the logical error rate of the code obtained through simulation.

III. SHUTTling BUS ARCHITECTURE SYNTHESIS FOR THE SURFACE CODE

The surface code [6–11] is a promising family of 2D topological stabilizer codes defined on a square lattice, featuring constant-weight (4-weight) stabilizer generators and an array of d^2 data qubits. The surface code has been widely studied for both spin qubits [27, 62], and other qubit technologies [11, 66–69], due to its 2D planar layout, high threshold values, and the existence of efficient decoding algorithms.

As shown in Figure 2, the distance-3 rotated surface code uses 9 data qubits (1 – 9) and 8 ancilla qubits ($X1 - X4$, and $Z1 - Z4$), each participating in weight-4 stabilizer checks that detect both bit-flip and phase-flip errors. During each QEC cycle, these stabilizers are measured in parallel to extract syndromes, which are fed into the decoder to maintain logical fidelity.

A. Chain Decomposition for Syndrome Extraction

For X and Z checks to be carried out in parallel while avoiding hook (or horizontal) errors [7], each check must be performed in a zig-zag pattern across the data qubits [9, 70, 71]. In this work we will use the interaction pattern shown in Figure 10 [72].

When composing the different interaction orders of each data qubit (from Figure 10) and grouping them into

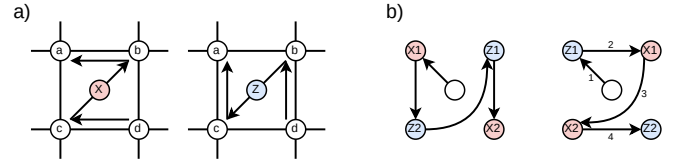


FIG. 3: Check interaction order to avoid hook (or horizontal) errors. **a)** Interaction order from the *check/ancilla qubit point of view* to avoid hook errors. **b)** Interaction order from the *data qubit point of view* to avoid hook errors. Note that the two figures have equivalent interactions order, depending on whether seen from the check/ancilla or the data qubits point of view. Numbers reference CNOT order as in Figure 2.

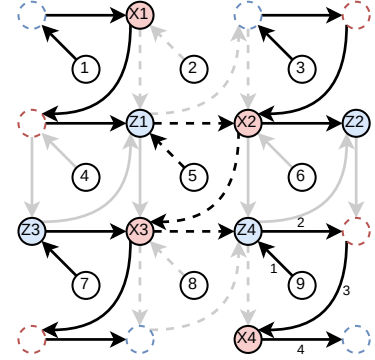


FIG. 4: Composite interaction order for the data qubits in the distance-3 surface code lattice. Empty, dashed qubits are shown at the patch boundaries to illustrate how the interaction structure extends when scaling to larger code distances.

the full surface code patch (see Figure 2) we obtain a directed graph representing all interactions between data and ancilla qubits for the syndrome extraction of the surface code, as it can be seen in Figure 4. It is important to notice that the aforementioned graph (called the *composite interaction directed graph*) has no cycles, which will be a critical property for the architecture synthesis method proposed in latter sections.

From the interaction pattern depicted in Figure 4 we can take, for each data qubit, the interactions with the different checks in the code and create what we call the *chain interactions*. These data-qubit-wise chain interactions (depicted in Figure 5) show what each data qubit is doing at each of the four slices of the syndrome extraction circuit. Note that data qubits at the border of the surface code patch have *idling* slices, where they are not interacting with any ancilla qubits from the code.

Due to the acyclic property of the composite interaction directed graph introduced before (see Figure 4) we can unify all the data qubit chains by merging nodes with the same label (*i.e.*, the ancilla qubits of the code).

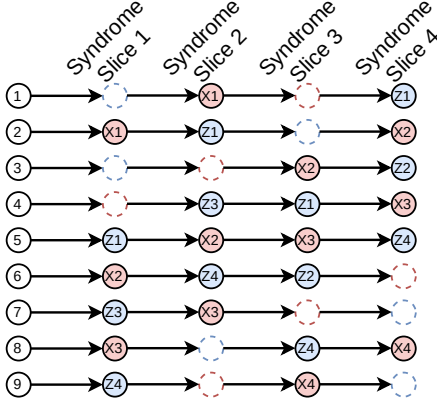


FIG. 5: Each data qubit’s interactions for each syndrome extraction slice in the distance-3 surface code.

By doing this we add to the ancilla qubits the relative order of interactions for data qubits (*e.g.*, data qubits $D1$ and $D2$ interact with $Z1$ after interacting with $X1$, and data qubit $D4$ interacts with $Z1$ after interacting with $Z3$) creating what we call the *chain-preserving directed acyclic graph* or chain-preserving DAG for short. Note that this graph can be constructed thanks to the acyclic property of the composite interaction graph, otherwise we would have cyclic dependencies on the order of the ancilla qubits, and the resulting chain-preserving graph would not be acyclic [73].

Figure 6 shows the chain-preserving DAG for the surface code with distance $d = 3$.

B. From the Surface Code to the Shuttling Bus Architecture

In designing the shuttling bus architecture, we must specify how storage and manipulation zones are arranged along the one-dimensional bus, and determine which data and ancilla qubits are assigned to each zone. While many possible assignments exist, the architecture must be designed so that qubit placement supports the surface code as robustly as possible, minimizing shuttling overhead and maximizing the performance of the logical encoding.

For this, and relying on other works dealing with the same architecture [62, 64], we only explore the placements of qubits that allow for the parallel shuttling of data qubits. With this constraint, which we call the *parallel shuttling constraint*, the shuttling bus will move all data qubits simultaneously into manipulation zones, that will be partly occupied by ancilla qubits.

One could ask the question whether a solution allowing for the parallel shuttling of all qubits exists, since the qubit interactions are given by a 2D code, and we are mapping it into a 1D placement. Fortunately, the chain-preserving DAG constructed before ensures that: each data qubit can be moved only in one direction (*i.e.*, always to the right or left of the shuttling bus) and will

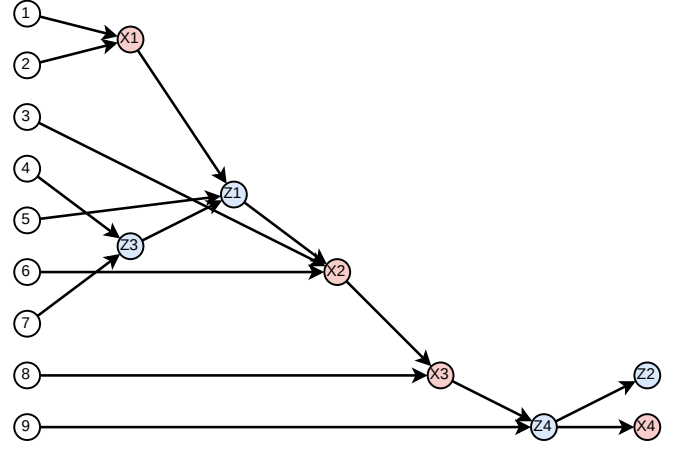


FIG. 6: Chain-preserving directed acyclic graph. Each incoming edge to a node represents a dependency on the shuttling bus placement, meaning that if node A is connected to node B ($A \rightarrow B$), A has to be placed left of B in the architecture.

meet all the ancilla qubit it should interact with. If a cycle was present on the composite interaction graph, that constraint would not be met, and a solution satisfying the parallel shuttling constraint would not exist.

To demonstrate the importance of this constraint, let us consider the following example. Consider a simple illustrative case with two data qubits $D1$ and $D2$, and two ancilla qubits A and B . $D1$ is scheduled to interact first with A and later with B . $D2$, on the other hand, is scheduled to interact first with B and later with A . If we translate these interaction orders into edges of the chain-preserving directed (acyclic) graph, we obtain a directed path $D1 \rightarrow A \rightarrow B$ from the first sequence and $D2 \rightarrow B \rightarrow A$ from the second one. Because the two paths from the graph point in opposite directions between A and B , they close into a cycle $A \rightarrow B \rightarrow A$, thus violating the acyclic property required by the chain-preserving DAG, meaning no parallel-shuttling schedule can satisfy all constraints in this example.

Since the above problem is not present in the surface code, any topological sort of the chain-preserving DAG created before results in a feasible placement of the qubits for the shuttling bus architecture, ensuring that all data qubits are moved in parallel to the right (or left) of the shuttling bus. A topological sort or topological ordering of a directed graph is a linear ordering of its vertices such that for every directed edge (u, v) from vertex u to vertex v , u comes before v in the ordering [74].

For this work, we (trivially) assume that data qubits will always be shuttled to the right, though this decision is not important since the results could be mirrored in the other direction.

As baseline for our work, we introduce the *Naive-topological* placement of the surface code into the shuttling bus architecture. For this naive layout, we compute a trivial topological sort of the chain-preserving DAG us-

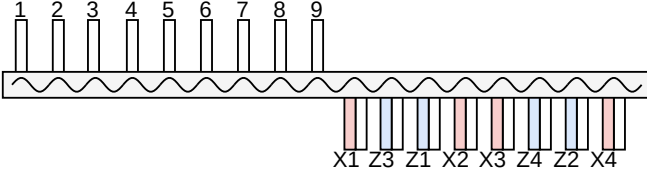


FIG. 7: *Naive-topological* placement for the distance-three surface code into the shuttling bus architecture.

ing Kahn's algorithm [75], and the resulting qubit order is then mapped into the architecture. Figure 7 shows the resulting layout for the surface code with distance $d = 3$.

C. Optimal Architecture Synthesis

We propose a mathematical model that synthesizes the physical architecture by optimally placing qubits in a one-dimensional shuttling bus. The model aims to obtain the qubit placement (*i.e.*, data and ancilla qubit ordering) that minimize both the total shuttling distance and the time for each error correction cycle.

Because of the parallel shuttling constraints posed before, all data qubits will be moved in between data-ancilla interactions to the right of the bus. So, at each timestep t , only the longest shuttle in that timestep will contribute to the overall latency, effectively hiding the latency of other, shorter, shuttles.

Let $\mathcal{C}$ be the chained interactions for a surface code, with the form $(d, a_1, a_2, a_3, a_4) \in \mathcal{C}$, where $d \in \mathcal{D}$ is a data qubit and $a_i \in \mathcal{A}$ is an ancilla qubit of the code. For a given positions $\mathcal{P}$ satisfying the parallel shuttling constraint (*i.e.*, $\mathcal{P}_d < \mathcal{P}_{a_1} < \mathcal{P}_{a_2} < \mathcal{P}_{a_3} < \mathcal{P}_{a_4}$) and allocating both the data and ancilla qubits ($\mathcal{Q} := \mathcal{D} \cup \mathcal{A}$), the total distance $D(\mathcal{P})$ to be shuttled for satisfying all interactions between qubits is given by:

$$D(\mathcal{P}) = \sum_{c \in \mathcal{C}} \sum_i \mathcal{P}_{c_{i+1}} - \mathcal{P}_{c_i} \quad (1)$$

$$= \sum_{(d, a_1, \dots) \in \mathcal{C}} (\mathcal{P}_{a_1} - \mathcal{P}_d + \quad (2)$$

$$\begin{aligned} & \mathcal{P}_{a_2} - \mathcal{P}_{a_1} + \\ & \mathcal{P}_{a_3} - \mathcal{P}_{a_2} + \\ & \mathcal{P}_{a_4} - \mathcal{P}_{a_3} + \\ & \mathcal{P}_{a_4} - \mathcal{P}_d) \\ &= \sum_{(d, a_1, \dots) \in \mathcal{C}} 2(\mathcal{P}_{a_4} - \mathcal{P}_d) \quad (3) \end{aligned}$$

For the same qubit allocation ($\mathcal{P}$), the distance contributing to time $T(\mathcal{P})$ (*i.e.*, the maximum distance to shuttle in each slice becomes the time bottleneck, since other shuttling latencies can be hidden due to the parallel shuttling employed) can be computed as:

$$T(\mathcal{P}) = \sum_i \max_{c \in \mathcal{C}} \mathcal{P}_{c_{i+1}} - \mathcal{P}_{c_i} \quad (4)$$

$$\begin{aligned} &= \max_{(d, a_1, \dots) \in \mathcal{C}} \mathcal{P}_{a_1} - \mathcal{P}_d + \\ & \max_{(d, a_1, \dots) \in \mathcal{C}} \mathcal{P}_{a_2} - \mathcal{P}_{a_1} + \\ & \max_{(d, a_1, \dots) \in \mathcal{C}} \mathcal{P}_{a_3} - \mathcal{P}_{a_2} + \\ & \max_{(d, a_1, \dots) \in \mathcal{C}} \mathcal{P}_{a_4} - \mathcal{P}_{a_3} + \\ & \max_{(d, a_1, \dots) \in \mathcal{C}} \mathcal{P}_d - \mathcal{P}_{a_4} \end{aligned} \quad (5)$$

We formulate the allocation problem as a Mixed-Integer Linear Program (MILP) problem [76, 77], in which the objective functions and all constraints are linear expressions, the positions of the qubits are encoded in a binary variable $x_{q,p} \in \{0, 1\}$, and time constraints for each slice are encoded in integer variables instead of integer variables we reduce the execution time of the solver.

Within this encoding, $x_{q,p} = 1$ when qubit $q \in \mathcal{Q}$ is allocated in position p , and $x_{q,p} = 0$ otherwise. Therefore, in order to obtain the position of a qubit q (previously $\mathcal{P}_q$), we now need to iterate through all the possible positions:

$$\mathcal{P}_q = \sum_{p \in \mathcal{P}} p x_{q,p} \quad (6)$$

The goal of the optimization problem is to concurrently minimize both the *time* and *distance* of the shuttles needed for satisfying all interactions in the syndrome extraction circuit. In the minimization function (see Equation (8)) each term is scaled by a hyperparameter (T and D respectively) to tune the time/distance tradeoff on the optimization problem.

Constraints in Equations (9) and (10) ensure that each zone contains exactly one qubit, and that each qubit is allocated into exactly one position. The parallel shuttling constraint, satisfying the chain dependencies between data and ancilla qubits, is satisfied by Equation (11). Lastly, constraints in Equations (12) to (16) are used to define the time for each shuttling slice ($t_1 - t_5$), for which its sum is minimized in the optimization function.

The MILP problem is formulated as follows:

$$\text{Optimize: } \min_{t,x} T \sum_i t_i + D \sum_{(d,a_1,\dots) \in \mathcal{C}} 2 \left(\sum_{p \in \mathcal{P}} p x_{a_4,p} - \sum_{p \in \mathcal{P}} p x_{d,p} \right) = \quad (7)$$

$$\min_{t,x} T \sum_i t_i + 2D \sum_{(d,a_1,\dots) \in \mathcal{C}} \sum_{p \in \mathcal{P}} p (x_{a_4,p} - x_{d,p}) \quad (8)$$

$$\text{Subject to: } \forall_{p \in \mathcal{P}} : \sum_{q \in \mathcal{Q}} x_{q,p} = 1 \quad (9)$$

$$\forall_{q \in \mathcal{Q}} : \sum_{p \in \mathcal{P}} x_{q,p} = 1 \quad (10)$$

$$\forall_{(d,a_1,\dots) \in \mathcal{C}} : \sum_{p \in \mathcal{P}} p x_{d,p} < \sum_{p \in \mathcal{P}} p x_{a_1,p} < \sum_{p \in \mathcal{P}} p x_{a_2,p} < \sum_{p \in \mathcal{P}} p x_{a_3,p} < \sum_{p \in \mathcal{P}} p x_{a_4,p} \quad (11)$$

$$\forall_{(d,a_1,\dots) \in \mathcal{C}} : t_1 \geq \sum_{p \in \mathcal{P}} p (x_{a_1,p} - x_{d,p}) \quad (12)$$

$$\forall_{(d,a_1,\dots) \in \mathcal{C}} : t_2 \geq \sum_{p \in \mathcal{P}} p (x_{a_2,p} - x_{a_1,p}) \quad (13)$$

$$\forall_{(d,a_1,\dots) \in \mathcal{C}} : t_3 \geq \sum_{p \in \mathcal{P}} p (x_{a_3,p} - x_{a_2,p}) \quad (14)$$

$$\forall_{(d,a_1,\dots) \in \mathcal{C}} : t_4 \geq \sum_{p \in \mathcal{P}} p (x_{a_4,p} - x_{a_3,p}) \quad (15)$$

$$\forall_{(d,a_1,\dots) \in \mathcal{C}} : t_5 \geq -1 \sum_{p \in \mathcal{P}} p (x_{d,p} - x_{a_4,p}) = \sum_{p \in \mathcal{P}} p (x_{a_4,p} - x_{d,p}) \quad (16)$$

Note that, in t_5 , each data qubit d in $(d, a_1, a_2, a_3, a_4) \in \mathcal{C}$ is shuttled from $\mathcal{P}_{a_4}$ to $\mathcal{P}_d$ (*i.e.*, from right to left in the shuttling bus). Since $\mathcal{P}_{a_4} > \mathcal{P}_d$ by Equation (11), to obtain the distance units for such shuttle we need to multiply by -1 the distance from $\mathcal{P}_{a_4}$ to $\mathcal{P}_d$, thus obtaining the right expression in Equation (16).

To solve the MILP problem and obtain the *Optimal* architecture placement, we use the Gurobi solver [78], which for linear programming problems uses several concurrent optimization algorithms such as the simplex [79, 80] or dual simplex algorithm [81], the branch and bound method [82], and interior-point (or barrier) methods [83, 84].

Because silicon spin qubits suffer from low dephasing times [85], we tune the proposed model so that it prioritizes time over distance in the optimization function by setting $T \gg D$. The resulting optimal shuttling bus architecture for the distance-three surface code is depicted in Figure 8.

Quantum Reverse Mapping for Other QEC Codes

The MILP formulation underlying the Quantum Reverse Mapping framework can be generalized to multiple

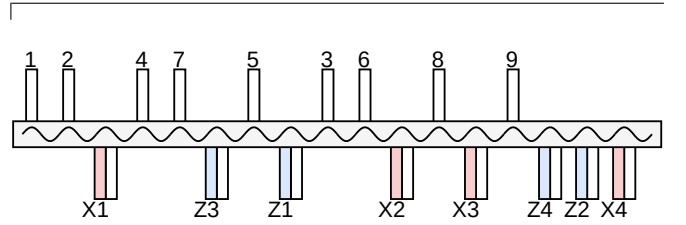


FIG. 8: *Optimal* placement for the distance-3 surface code into the shuttling bus architecture.

families of QEC codes, since it does not rely on any code-specific structure beyond the scheduling constraints of the syndrome-extraction circuit. The only requirement for applying the MILP model to an arbitrary quantum error-correcting code is that the ancilla-data interactions used during stabilizer measurement can be represented as a directed acyclic graph (DAG). This ensures that all qubits participating in a stabilizer measurement can be shuttled in the same direction in parallel without introducing cyclic dependencies in the interaction chain (see Figure 6).

Once the DAG is available, adapting the optimization model to a new code involves two straightforward modifications. The ordering constraints that enforce $\mathcal{P}_d < \mathcal{P}_{a_1} < \mathcal{P}_{a_2} < \dots$ (Equation (11) for the surface code) must be adjusted according to the ancilla-data in-

teraction chains of the new QEC code. And, the number of time-slice variables (t_i) must be adjusted to the maximum stabilizer weight. For the rotated surface code, all stabilizers have weight 4 (with lower-weight boundary checks), so each measurement circuit decomposes into four interaction slices plus a final return step, resulting in five shuttling intervals. Codes with higher-weight stabilizers (e.g., weight-6 color-code checks) naturally require more time variables.

No other part of the MILP formulation requires modification. All objective terms and constraints related to minimizing total shuttling distance and cycle duration remain identical.

To illustrate the generality of the framework, Appendix A applies it to the family of 2D color codes [86, 87], a prominent alternative to the surface code due to their lower qubit overhead and ability to perform transversal Clifford gates. These codes feature weight-6 stabilizers and require adapted scheduling and chain extraction, but remain fully compatible with the DAG-based modeling approach.

D. Heuristic Architecture Synthesis

As the code distance increases, the size of the mathematical optimization model proposed in the previous section grows substantially, both in terms of variables and constraints, making it impractical to solve exactly within a reasonable time frame for larger systems.

To quantify this complexity, consider that for a surface code with distance d , the system comprises $N = d^2 + (d^2 - 1) = O(d^2)$ qubits. The resulting MILP formulation requires defining the problem state using 5 integer variables (t_i) and N^2 binary variables $x_{q,p}$ (encoding the mapping of qubits to physical sites). Furthermore, the model necessitates $6N$ linear constraints to ensure correctness. While the number of constraints grows quadratically with d , the number of binary variables scales with the fourth power of d ($N \times N = O(d^4)$). Crucially, because the worst-case time complexity of general MILP solvers is exponential with respect to the number of variables (scaling as $O(2^{N^2}) = O(2^{d^4})$ [88]), this explosion in the search space creates a severe bottleneck for exact solvers as the code distance increases.

Therefore, in order to handle higher code distances, we need a scalable, computationally efficient method. A straightforward option is to generate the topological ordering of qubits using the chain-preserving DAG, the *Naive-topological* placement introduced in earlier sections.

Taking inspiration from the solution obtained using the mathematical model, what we call the *Optimal* placement, we devise a heuristic strategy to place the qubits with the goal of obtaining a similar performance to the *Optimal* placement, while maintaining the computational complexity of the *Naive-topological* one.

To achieve this, we propose the *Zig-Zag* placement.

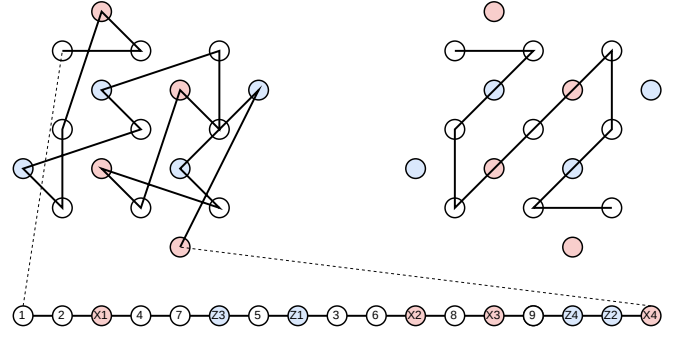


FIG. 9: Data qubits order (right) following the *zig-zag* order. The left and bottom diagrams depict the order of all qubits from the surface code, adding ancilla qubits as soon as they are available while satisfying the topological order given by the chain-preserving DAG.

This architecture synthesis strategy starts by placing the top left data qubit of the surface code (*i.e.*, data qubit 1 in Figure 2), and continue by placing the other data qubits in a zig-zag pattern, as depicted in Figure 9 (right). Ancilla qubits are placed as soon as they are available satisfying the topological order given by the chain-preserving DAG. The pseudo-code for the proposed synthesis method is given in Algorithm 1.

For the distance-3 surface code, the *Zig-Zag* placement strategy exactly matches the *Optimal* one, resulting in the architecture depicted in Figure 8.

Algorithm 1 Zig-Zag Placement Synthesis

Require: Code distance d
Ensure: Qubit Placement $\mathcal{P}$

1. **Initialization**
- 1: $\mathcal{D}, \mathcal{A} \leftarrow \text{GenerateSurfaceCode}(d)$
- 2: $G \leftarrow \text{ChainPreservingDAG}(\mathcal{D}, \mathcal{A})$
2. **Construct Zig-Zag Order**
- 3: $L \leftarrow$ Data qubits $\mathcal{D}$ ordered in the zig-zag pattern
3. **Placement Loop**
- 4: $\mathcal{P} \leftarrow \emptyset$
- 5: **for** each q_d in L **do**
- 6: Append q_d to $\mathcal{P}$
- 7: Remove q_d from G
- 8: $\mathcal{A}' \leftarrow \{a \in \mathcal{A} \mid \text{InDeg}(G, a) = 0\}$
- 9: **while** $\mathcal{A}' \neq \emptyset$ **do**
- 10: Sort $\mathcal{A}'$ by priority (connectivity weight)
- 11: **for** q_a in $\mathcal{A}'$ **do**
- 12: Append q_a to $\mathcal{P}$
- 13: Remove q_a from G
- 14: **end for**
- 15: **end while**
- 16: **end for**
- 17: **return** $\mathcal{P}$

The computational complexity of the proposed Zig-Zag placement strategy is dominated by the number of qubits N , scaling linearly as $O(N)$ or equivalently $O(d^2)$ with respect to the code distance d . The construction of the L order requires visiting each data qubit exactly once, an

$O(N)$ operation. By maintaining a map of data-ancilla dependencies (*i.e.*, which ancilla qubits (≤ 4) depend on each data qubit) checking and updating the set of available ancillas ($\mathcal{A}'$) inside the main loop can be performed in constant time, independent of the total system size. Consequently, as the placement loop iterates through the data qubits exactly once and performs only local, constant-time updates, the overall computational cost of the synthesis algorithm is $O(N) = O(d^2)$.

E. Comparing the Optimal and Heuristic Architecture Synthesis

The three proposed architecture synthesis methods, *Naive-topological*, *Zig-Zag*, and *Optimal* placement, only compute the relative positions of the qubits based on the interactions, but the methods are agnostic of the exact timing of each interaction. Since some data qubits placed at the edge of the surface code patch are idling at some syndrome extraction slices, when putting together the chain interactions in the synthesized architecture we may need to add extra spaces for the data qubits to be stored when not interacting. Extra quantum dots to hold the spins in idling slices are added greedily upon necessity.

The *Optimal* and *Zig-Zag* placements alternates data and ancilla qubits in the shuttle bus, creating many feasible positions for each idling data qubit to be stored in. However, the *Naive-topological* method places all data qubits at the beginning (left) of the shuttle bus array, thus not having free intermediate positions for the idling data qubits, thus necessitating more extra quantum dots.

Figures 10a and 10b show the shuttles needed of each data qubit across the syndrome extraction circuit for the surface code with distance $d = 3$ using the *Naive-topological* and *Optimal* placements respectively. After the last interaction, each data qubit is returned to its original position (shuttle not shown in the figure). Since (for the distance-three surface code) the placements obtained with the *Optimal* and *Zig-Zag* strategies are the same, the shuttlings and interactions between both placements are also the same (see Figure 10b).

The proposed heuristic *Zig-Zag* placement coincides with the *Optimal* one for low-distance codes. Because of the high computational cost of the mathematical optimization model, we cannot obtain the *Optimal* placement for code distances equal or higher than $d = 9$. Therefore, we cannot assess the optimality of the heuristic method for higher code distances. Nonetheless, we can characterize the scaling trends, in terms of shuttling distance, shuttling time, and shuttle bus size (accounting for the extra quantum dots) for the different proposed placement techniques.

To understand the efficacy of the *Zig-Zag* heuristic, it is useful to view the problem through the lens of local neighborhood preservation. In the Surface Code grid, data qubits that share a common ancilla are always located at distance < 2 (immediate spatial neighbors). The

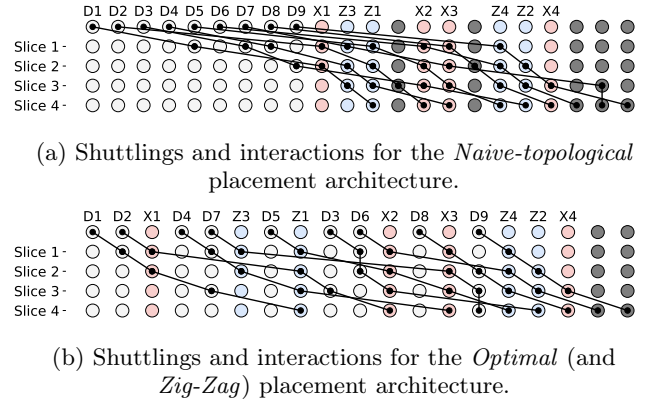


FIG. 10: *Naive-topological* and *Optimal* (and *Zig-Zag*) architectures for the surface code with distance $d = 3$. Note that five extra quantum dots are needed for the *Naive-topological* placement, which grows with code distance. On the other hand, for the *Optimal* placement (and also for the *Zig-Zag* one) only two extra quantum dots are needed, which is constant across all code sizes.

heuristic aims to preserve this distance-1 connectivity in the linear layout. For example, in a distance-3 code, starting with data qubit 1 at position $(0, 0)$, the set of immediate neighbors is $\{(1, 0), (0, 1), (1, 1)\}$ (*i.e.* data qubits 2, 4, and 5). By prioritizing the placement of $(1, 0)$ next to $(0, 0)$, the heuristic ensures that the X-type ancilla shared between them (qubit X1 in the example) can be executed with minimal shuttling overhead. Subsequently, the heuristic targets the next available distance-1 neighbor, $(0, 1)$, prioritizing it over $(1, 1)$ to minimize the number of ancilla qubits added to the 'pending' set (those not yet placed but whose data qubits are already in the placement) thereby preventing the high shuttling distances associated with maintaining these long-range dependencies. This recursive greedy selection of immediate grid neighbors generates the zig-zag pattern. While this is not a formal proof of optimality for all code distances, our MILP results confirm that this geometric strategy yields mathematically optimal placement for code distances $d = 3, 5, 7$.

Figure 11 depicts the scaling trends for the different proposed architectures, considering code sizes from $d = 3$ to $d = 21$. The results confirm the *zig-zag* synthesis methodology as a promising candidate, matching the performance of the *optimal* one (for code distance 3, 5, and 7) while maintaining the computational cost of the *Naive-topological* one.

IV. SURFACE CODE SIMULATION IN A SHUTTLE BUS ARCHITECTURE

Beyond evaluating architectural metrics related to scalability, it is essential to assess how well the proposed architecture performs in its intended role: reliably stor-

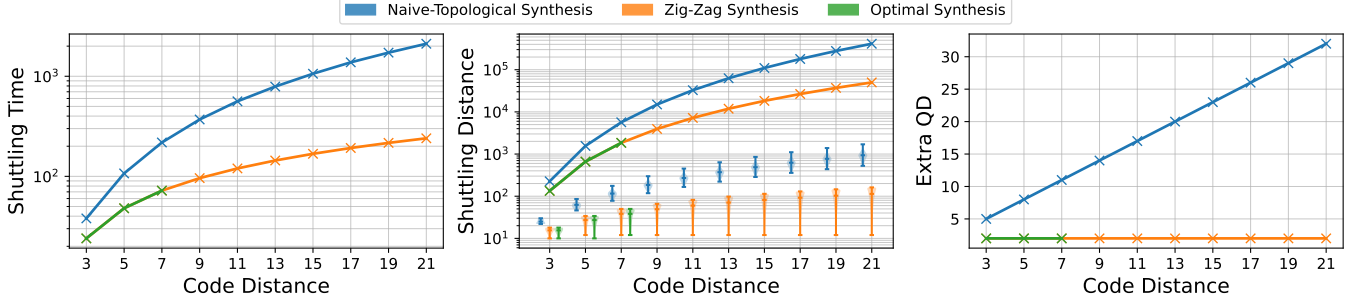


FIG. 11: Shuttling time (left), shuttling distance (middle), and added number of extra quantum dots (right) when increasing code distance. Shuttling time (left) can be decomposed into the time of the four different shuttles (one per each slice) and the time of the return shuttle which together form an error correction cycle. The total shuttling time scales linearly for the *Optimal* and *Zig-Zag* architectures, and quadratically for the *Naive-topological* one. Shuttling distance (middle) scales polynomially for all the placement methods. Violin plots depict the variability of the shuttling distance across all data qubits. The number of extra quantum dots (right) that need to be added is constant (2) for the *Optimal* and *Zig-Zag* architectures, and linear for the *Naive-topological* one.

ing a logical qubit using the surface code.

In this section we simulate the surface code memory circuit under realistic noise conditions on the shuttling bus architecture using the *Zig-Zag* placement. For the simulation we use the Stim library [72, 89], which has been proven to be an efficient and scalable library for stabilizer circuits simulation.

A. Noise Model

We assume three sources of error contributing to the noise added to the quantum states, dividing the error sources into idling, gate, and shuttling-induced.

Under realistic conditions, dephasing is expected to be the dominant form of decoherence for silicon spin qubits [85]. Spins that are idling in a quantum dot will be decohered according to T_2^* , modeled as a phase damping channel [3]. Therefore, a quantum state ρ that stays for t seconds in a quantum dot with decoherence time $T_{2,QD}^*$, will be decohered according to:

$$\mathcal{N}(\rho) \rightarrow E_0 \rho E_0^\dagger + E_1 \rho E_1^\dagger \quad (17)$$

With $E_0 = \begin{bmatrix} 1 & 0 \\ 0 & \sqrt{1-\lambda} \end{bmatrix}$, $E_1 = \begin{bmatrix} 0 & 0 \\ 0 & \sqrt{\lambda} \end{bmatrix}$, and $\sqrt{1-\lambda} = e^{-t/2T_{2,QD}^*}$, which can be modeled with a Z-Pauli $\mathcal{E}_p(\rho)$ channel [3] by setting:

$$p = \frac{1 - e^{-t/2T_{2,QD}^*}}{2} \quad (18)$$

Allowing us to add it into the Stim simulation, which only accepts Pauli channels as noise sources.

In our simulations, we assume gate times of 100 ns for single-qubit operations and 50 ns for two-qubit operations [48, 63, 85, 90]. While in practice single- and two-qubit gates often exhibit different fidelities, we base

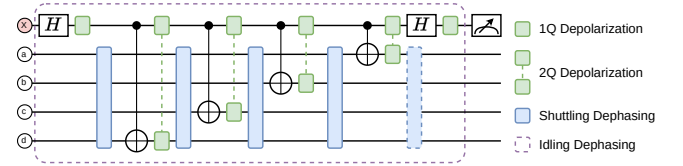


FIG. 12: Noisy circuit for the X-Checks of the surface code.

our model on state-of-the-art results where their performance is comparable, and approximate both as depolarizing channels with parameter p . For qubit readout, we assume a duration of 500 ns, reflecting an optimistic yet experimentally supported estimate [91–94], and neglect state-preparation-and-measurement (SPAM) errors. We emphasize that this corresponds to an optimistic assumption for both gate and readout performance, and in the following sections we separately explore the effect of depolarizing noise and coherence times.

It is still unclear how errors due to shuttling affect qubits, though several works assessed it from both theoretical and experimental perspectives [48, 85, 95].

We take the experimental noise characterization obtained in [95] and model the shuttling noise as a phase damping channel, similar than for the T_2^* time mentioned before. However, as described in [95], the effective dephasing time for the shuttled states depends on the base dephasing time of the shuttling bus ($T_{2,bus}^*$), the distance of the shuttle d_{sh} , and the quantum dot correlation length l_c , resulting in an effective dephasing time $T_{2,sh}^*$ for the shuttling given by:

$$T_{2,sh}^* = T_{2,bus}^* \sqrt{\frac{d_{sh} + l_c}{l_c}} \quad (19)$$

which will also be modeled as a phase damping (Z-Pauli) noise channel using Equation (18).

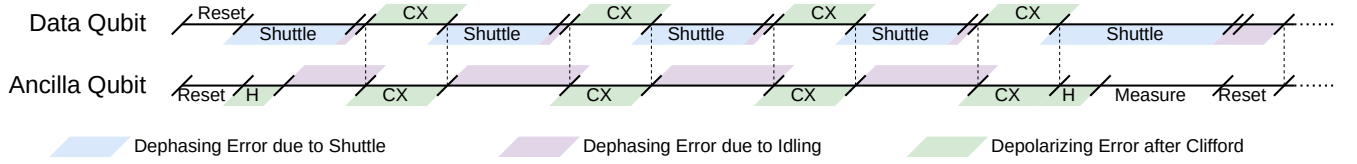


FIG. 13: Syndrome extraction cycle breakdown for data and ancilla qubits. Including synchronization steps (vertical dashed lines) and noise applied to each one of the steps.

Parameter	Value
Correlation Length (l_c)	13 nm [48]
Distance Between Qubits (d_{qu})	100 nm
Shuttling Velocity (v_{sh})	2.8 m/s [95]
Single-Qubit Gate Time	100 ns
Two-Qubit Gate Time	50 ns
Measurement Time	500 ns
Gate Depolarization (p)	0, 0.001, 0.005
T_2^* Quantum Dot (T_{2QD}^*)	100 μ s, 1000 μ s, $+\infty$
T_2^* Shuttling Bus (T_{2bus}^*)	[1 μ s, 100 μ s], $+\infty$
Code Distance	3 – 9
Synthesis Method	Zig-Zag Placement

TABLE I: Simulation parameters.

Figure 12 depicts the error locations for the X parity checks.

B. Noise Threshold Calculation

We now want to evaluate how much noise the architecture can tolerate from each one of the three noise sources introduced before. Figure 13 depicts the steps needed for the syndrome extraction for both the data and ancilla qubits. It also shows which quantum gates need to be synchronized (vertical dashed lines), and what type of noise is applied at each step.

Table I summarizes the simulation parameters. Note that p (depolarizing noise), T_{2QD}^* , and T_{2bus}^* are set to 0, $+\infty$, and $+\infty$, respectively, since we begin the simulations by analyzing independently each noise source.

To do so, we set two of the three noise sources to loss-less values and sweep the third one for different code sizes, setting the number of correction rounds to three times the code distance. From the obtained logical error rate per round, we compute the pseudo-threshold as the mean crossing point across all code distances, also taking into account the standard deviation in the crossings.

Figures 14, 15, and 16 show the logical error rate per round for code distances 3 to 9 when exploring the three different noise sources considered. These figures also report the mean and deviation of the pseudo-threshold.

We can see how, when exploring the different noise sources, the threshold deviation (*i.e.*, whether or not all logical rates distances cross at the same point) varies significantly, specially for the idling dephasing time T_{2QD}^* .

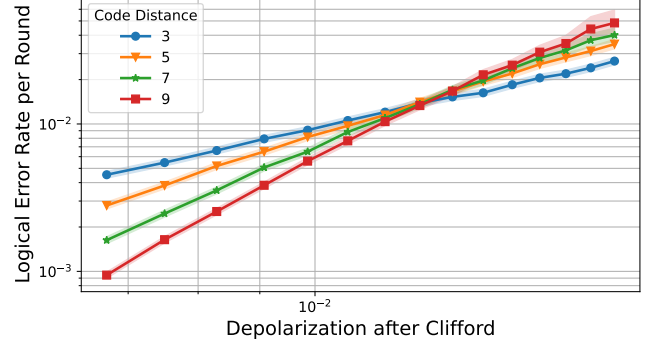


FIG. 14: Gate depolarizing noise threshold calculations. Threshold value: $0.012293 \pm 6.35 \times 10^{-4}$

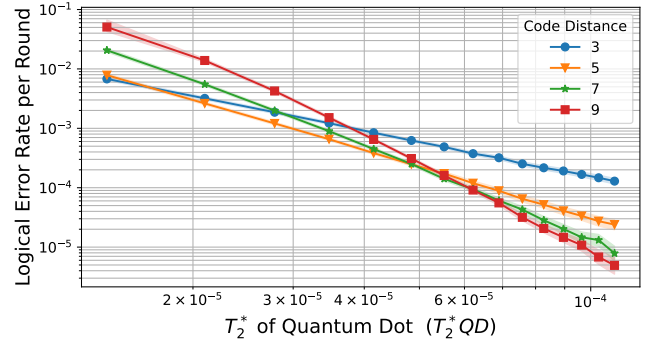


FIG. 15: T_{2QD}^* of the Quantum Dot noise threshold calculations. Threshold value: $4.94469 \times 10^{-5} \pm 7.74 \times 10^{-6}$

When assessing each noise source independently, we obtain upper bounds on how much can the architecture tolerate from each noise source. However, all noises are present in a realistic architecture, and therefore we must assess how the different noises combine together.

Since in this work we mainly focus on the shuttling process and its impact on the overall logical error rate, we study how much T_{2bus}^* can the architecture tolerate for some fixed depolarization (p) and dephasing times (T_{2QD}^*). The results are depicted in Figure 17, including, in the bottom row, the threshold trends for the different noise combinations considered.

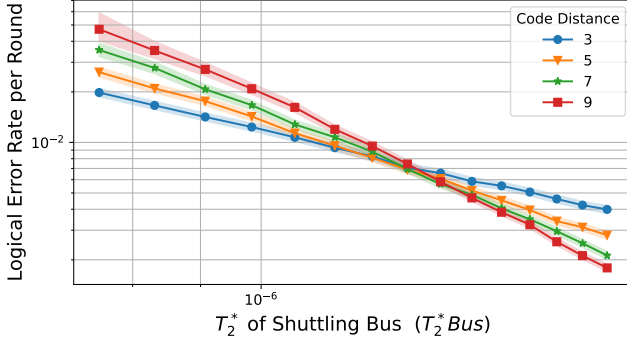


FIG. 16: T_2^* of the Shuttling Bus noise threshold calculations. Threshold value: $1.375675 \times 10^{-6} \pm 7.2 \times 10^{-8}$

In the next sections, we explore the impact of the architecture design decisions in the performance of the error correction code by fixing the noise parameters to values above the threshold for the architecture parameters considered until now.

C. Designing Shuttling Bus Architectures

The previously explored parameters (*i.e.*, the physical noise sources) are intrinsic to the underlying technology and not adjustable by architecture designers. In this section, we shift focus to architectural decisions and analyze their impact on the logical error rate. To isolate these effects, we fix the noise parameters to $p = 0.001$ (corresponding to a gate fidelity of 99.925%), $T_{2QD}^* = 100\mu s$, and $T_{2bus}^* = 10\mu s$, and explore how the distance between the elements in the shuttling bus d_{qu} (previously set to $d_{qu} = 100nm$) and the shuttling velocity v_{sh} (previously $v_{sh} = 2.8m/s$) affect performance. The value for T_{2QD}^* was chosen under the assumption that quantum dots $\{Q_0, \dots, Q_n\}$ (see Fig. 1) do not have micromagnets nearby, which leads to higher T_{2QD}^* due to low influence of charge noise on dephasing [94, 96]. We assume that the micromagnets are present only near the dots $\{O_0, \dots, O_n\}$, so by shuttling towards any of these dots, in its vicinity, the qubit will experience the magnetic field gradient together with the noise induced by shuttling itself, which as we assume leads to smaller values of T_{2bus}^* . However, we do vary T_{2QD}^* and T_{2bus}^* in a wide range of values and study their effect on the code performance.

Because of the effect in T_{2sh}^* of motional narrowing when shuttling across longer distances [95] (see Equation (19)), both parameters d_{qu} and v_{sh} do not have a linear relationship, and each combination of parameters must be tested independently.

Figure 18 depicts the logical error rate when exploring the distance between qubits (d_{qu}) for four different shuttling velocities, ranging from 1 to 10 m/s. The bot-

tom row shows the threshold for each shuttling velocity computed as the mean among the intersections, with the shaded region representing the standard deviation.

As expected, a higher shuttling velocity leads to a higher tolerance for the distance between qubits. In the considered error model, there is no penalization for high shuttling velocities or low qubit distances. However, technology will limit the values that both parameters can take.

To further explore the impact of the two considered architectural parameters, we explore the logical error rate per round obtained for a vast combination of shuttling velocities and qubit distances. Figure 19 shows the logical error rate per round for code distance 3, 5, 7, and 9 when exploring architectural parameters. The red and black lines depict the isoclines on logical error rate and the threshold fitted from the data in Figure 18, respectively.

As expected, when increasing the code distances, the logical error rate decreases for all the architecture configurations above the fitted threshold (dashed black line), and increases for the configurations below the fitted threshold.

D. Suboptimal Architecture Synthesis

After assessing, in the previous sections, the impact of the noise parameters and the architectural decisions on the performance of the surface code, in this section we explore the logical error rate for the three different architecture synthesis techniques discussed in Section III.

The noise parameters are set as in the previous experiments ($p = 0.001$, $T_{2QD}^* = 100\mu s$, and $T_{2bus}^* = 10\mu s$), and we explore the three following architecture configurations:

- $d_{qu} = 200nm$ and $v_{sh} = 5m/s$.
- $d_{qu} = 100nm$ and $v_{sh} = 10m/s$.
- $d_{qu} = 100nm$ and $v_{sh} = 20m/s$.

with the three proposed architecture synthesis techniques (*Naive-topological*, *Zig-Zag*, and *Optimal*).

Figure 20 shows the logical error rate per round when increasing the code distance for the three architecture configurations, and the three considered architecture synthesis techniques. Due to the computational cost of obtaining the *Optimal* placement, we limit the exploration of such architecture up to code distance 7. For the *Naive-topological* and *Zig-Zag* placements we explore up to code distance 21, achieving logical error rates up to 2×10^{-10} .

The results obtained highlight the importance of synthesizing the optimal or close to optimal layout, since, for the same placement configuration and noise values, the *Optimal* and *Zig-Zag* placements obtain better logical error rates when increasing the code distance, while the *Naive-topological* placement obtain worse logical error rates when increasing the code distance, meaning that

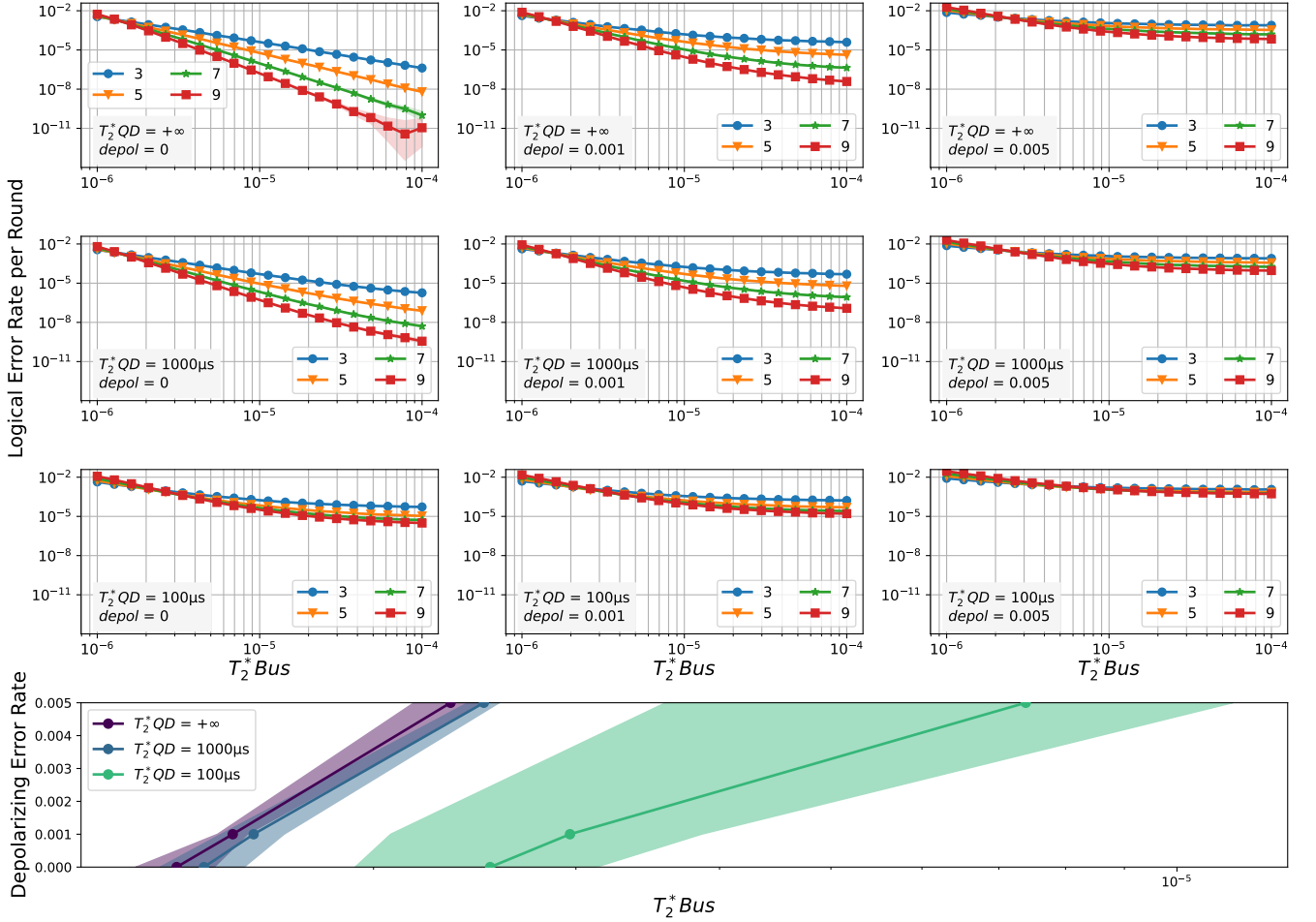


FIG. 17: T_2^* of the Shuttling Bus noise threshold calculations under several configurations of depolarizing noise after each gate, and T_2^* of the Quantum Dot.

the architecture configuration may be below the threshold.

With this exploration we conclude the analysis of the proposed architecture and highlight the under performance of the *Naive-topological* layout synthesis, while showcasing the equal performance of the *Optimal* and *Zig-Zag* synthesis for systems up to code distance 7. Yet, we believe the *Zig-Zag* placement matches the optimal one for larger code distances, but due to the computational cost of obtaining such optimal layouts this remains an unanswered question.

V. CONCLUSIONS

We introduced a complete methodology for synthesizing an optimal architecture for the rotated surface code on a one-dimensional spin-qubit shuttling bus. Our approach begins with a mixed-integer linear program (MILP) that precisely models the placement and shuttling constraints for syndrome extraction. This formu-

lation yields exact solutions with relatively low execution time for code distances below $d = 9$, allowing us to explore the structure of optimal encodings. Based on this structure, we proposed the *Zig-Zag* heuristic for the placement of the rotated surface code, which scales linearly with the number of qubits and maintains the optimal cost in terms of total shuttling distance and cycle latency.

The MILP formulation at the core of the Quantum Reverse Mapping framework is not specific to the rotated surface code. The only requirement for applying our method to any quantum error-correcting code is that its syndrome-extraction interactions can be represented by a directed acyclic graph. Once this DAG is provided, the same optimization model directly applies, making the framework broadly general and adaptable to a wide range of error correction codes.

We assessed the three proposed architecture synthesis methods with detailed architectural metrics. Compared to a naive-topological placement, our methods reduce the total shuttling distance and round duration from

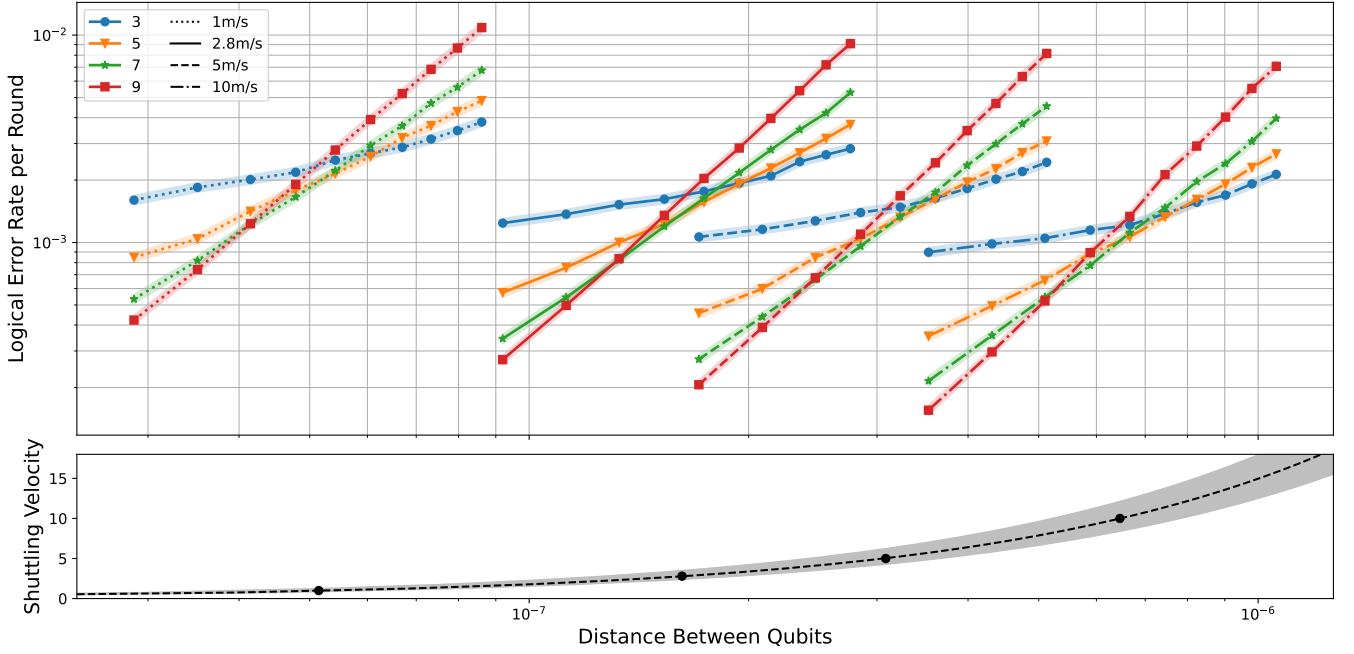


FIG. 18: Logical error rate (top row) for four different shuttling velocities (1 – 10 m/s) for several distance between qubits in the shuttling bus. The lower row represents the threshold (mean and standard deviation) for the shuttling velocity / distance between qubits tradeoff.

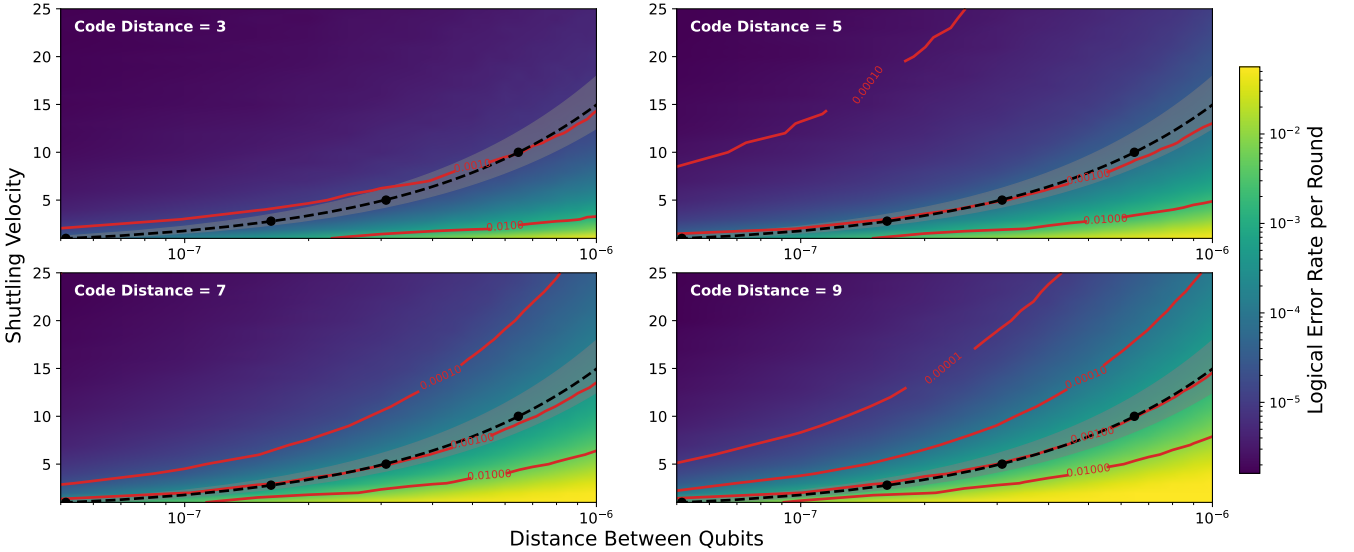


FIG. 19: Logical error rate per correction round for distances 3, 5, 7, and 9 for different shuttling velocities, distance between qubits configurations. Red lines represent the isoclines for the logical error rate and the dashed black line depicts the threshold fit.

quadratic to linear scaling with respect to the code distance. Additionally, our approach requires only two extra quantum-dot storage sites irrespective of the code size, whereas the naive method incurs spatial overhead linear to the code distance d .

To evaluate fault tolerance, we performed simulations

using Stim under a noise model combining gate depolarization, quantum-dot dephasing, and shuttling-induced decoherence. We observed that, for realistic spin-qubit parameters, the synthesized architecture achieves logical error rates below 2×10^{-10} per round at code distance 21. These results demonstrate that spin-based quantum

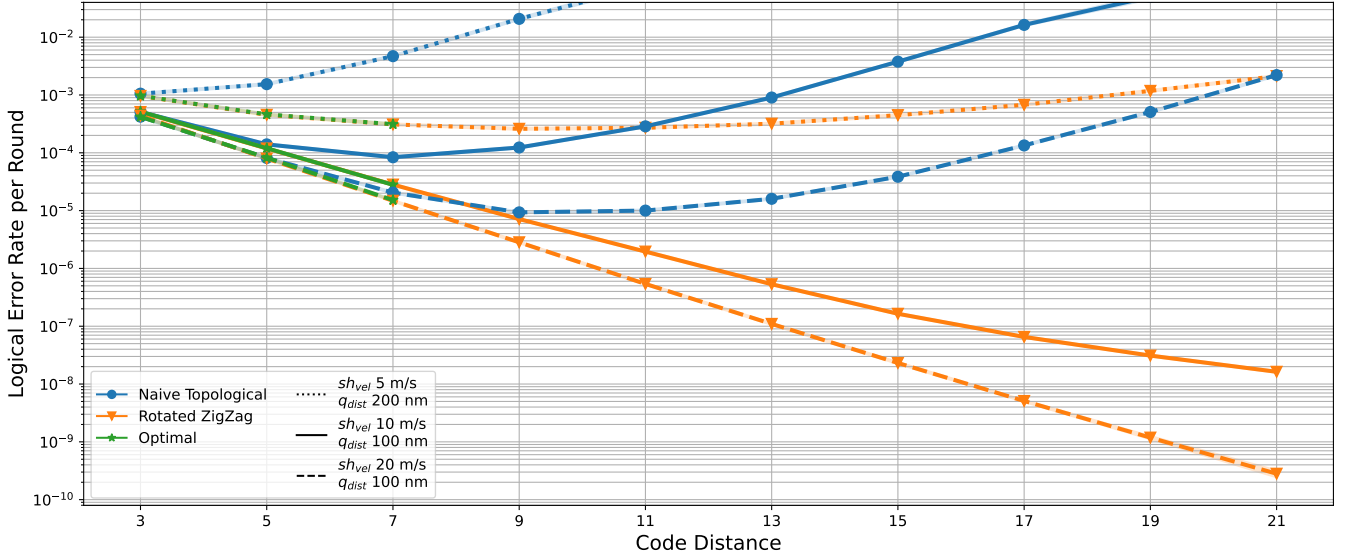


FIG. 20: Logical error rate per round for the three proposed architecture synthesis techniques under three architecture configurations. Noise values are set to $p = 0.001$, $T_{2,QD}^* = 100\mu s$, and $T_{2,bus}^* = 10\mu s$.

platforms are approaching the fidelity and coherence required for scalable fault-tolerant computation using surface codes.

This work underscores the importance of optimally encoding a single logical qubit before scaling up to large architectures. The reverse-mapping shuttling-bus architecture design provides a robust foundation upon which higher-level heuristic compilers can operate, ensuring that the scaling of quantum computers is supported by the strongest possible logical layer.

ACKNOWLEDGMENTS

Authors acknowledge funding from the EC through HORIZON-EIC-2022-PATHFINDEROPEN-01-101099697 (QUADRATURE). EA acknowledges funding from the Spanish Ministry of Science, Innovation and Universities and European ERDF (QCOMM-CAT-Planes Complementarios), Generalitat de Catalunya, and the ICREA Academia Award 2024. SA acknowledges funding from the EC through HORIZON-ERC-2021-101042080 (WINC). EB acknowledges funding from Science Foundation Ireland under Grant 21/RP-2TF/10019. CGA acknowledges funding from the QuantERA grant EQUIP with the grant number PCI2022-133004, funded by Agencia Estatal de Investigación, Ministerio de Ciencia e Innovación, Gobierno de España, MCIN/AEI/10.13039/501100011033, and by the European Union NextGenerationEU/PRTR, and from the Spanish Ministry of Science, Innovation and Universities and European ERDF under grants PID2021-123627OB-C51 and PID2024-158682OB-C31. PE acknowledges funding from the predoctoral FPI-UPC

grant supported by UPC and Banco Santander.

DATA AVAILABILITY

All code developed and used in this work, including the optimization framework, architecture synthesis routines, and quantum error-correction simulations, is publicly available on Zenodo at <https://zenodo.org/records/17405430> [97].

Appendix A: Applicability to Color Codes

In this work we propose a methodology based on the formulation of a MILP optimization problem to synthesize an optimal architecture for the surface code in a shuttling bus architecture. However it may be unclear which of the proposed methods can be applied to other QEC codes and which ones are intrinsic of the surface code.

The only claim only applicable to the surface code is that the *Zig-Zag* synthesis method has an optimal performance (at least for code distances up to $d = 7$). The other proposed architecture synthesis techniques (*i.e.*, the *Naive-topological* and the *Optimal* one using the MILP formulation) can be directly applied to any QEC code for which we can construct the chain-preserving DAG (see Figure 6 from the main text), which not only depend on the code, but also on the scheduling of the data-ancilla interactions.

To illustrate the generality of the MILP-based Quantum Reverse Mapping framework, we extend the approach to the family of 2D color codes [86, 87]. For these codes, we obtain both the lattice construction and

the data-ancilla interaction ordering for the syndrome-extraction circuit from [98]. This interaction structure naturally produces an acyclic dependency graph, fulfilling the only requirement needed to apply the MILP model.

The adaptation to color codes is therefore minimal: because their stabilizers have weight six, the syndrome-extraction circuit requires six interaction slices instead of the four used in the rotated surface code. Consequently,

the number of time variables in the MILP increases from five to seven (six forward slices plus a return step), while all other constraints remain unchanged.

Using this extended formulation, we synthesize optimal one-dimensional shuttling architectures for 2D color codes with distances $d = 3$, $d = 5$, and $d = 7$. The resulting stabilizers and optimal placements are shown in Figures 21, 22, and 23, respectively.

-
- [1] P. W. Shor, Scheme for reducing decoherence in quantum computer memory, *Phys. Rev. A* **52**, R2493 (1995).
 - [2] J. Roffe, Quantum error correction: an introductory guide, *Contemporary Physics* **60**, 226 (2019).
 - [3] M. A. Nielsen and I. L. Chuang, *Quantum computation and quantum information* (Cambridge university press, 2010).
 - [4] C. Gidney and M. Ekerå, How to factor 2048 bit rsa integers in 8 hours using 20 million noisy qubits, *Quantum* **5**, 433 (2021).
 - [5] C. Gidney, How to factor 2048 bit rsa integers with less than a million noisy qubits, arXiv preprint arXiv:2505.15917 (2025).
 - [6] S. B. Bravyi and A. Y. Kitaev, Quantum codes on a lattice with boundary, arXiv preprint quant-ph/9811052 (1998).
 - [7] E. Dennis, A. Kitaev, A. Landahl, and J. Preskill, Topological quantum memory, *Journal of Mathematical Physics* **43**, 4452 (2002).
 - [8] A. G. Fowler, A. M. Stephens, and P. Groszkowski, High-threshold universal quantum computation on the surface code, *Physical Review A* **80**, 052312 (2009).
 - [9] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, Surface codes: Towards practical large-scale quantum computation, *Physical Review A* **86**, 032324 (2012).
 - [10] D. Litinski, A game of surface codes: Large-scale quantum computing with lattice surgery, *Quantum* **3**, 128 (2019).
 - [11] Suppressing quantum errors by scaling a surface code logical qubit, *Nature* **614**, 676 (2023).
 - [12] R. Versluis, S. Poletto, N. Khammassi, B. Tarasinski, N. Haider, D. J. Michalak, A. Bruno, K. Bertels, and L. DiCarlo, Scalable quantum circuit and control for a superconducting surface code, *Physical Review Applied* **8**, 034021 (2017).
 - [13] S. Krinner, N. Lacroix, A. Remm, A. Di Paolo, E. Genois, C. Leroux, C. Hellings, S. Lazar, F. Swiadek, J. Hermann, *et al.*, Realizing repeated quantum error correction in a distance-three surface code, *Nature* **605**, 669 (2022).
 - [14] D. Pataki and A. Pályi, Compiling the surface code to crossbar spin qubit architectures, *Physical Review B* **111**, 115307 (2025).
 - [15] M. Veldhorst, H. G. Eenink, C.-H. Yang, and A. S. Dzurak, Silicon cmos architecture for a spin-based quantum computer, *Nature communications* **8**, 1766 (2017).
 - [16] B. Hetényi and J. R. Wootton, Tailoring quantum error correction to spin qubits, *Physical Review A* **109**, 032433 (2024).
 - [17] M. Y. Siraichi, V. F. d. Santos, C. Collange, and F. M. Q. Pereira, Qubit allocation, in *Proceedings of the 2018 international symposium on code generation and optimization* (2018) pp. 113–125.
 - [18] G. Li, Y. Ding, and Y. Xie, Tackling the qubit mapping problem for nisq-era quantum devices, in *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems* (2019) pp. 1001–1014.
 - [19] S. Niu, A. Suau, G. Staffelbach, and A. Todri-Sanial, A hardware-aware heuristic for the qubit mapping problem in the nisq era, *IEEE Transactions on Quantum Engineering* **1**, 1 (2020).
 - [20] P. Escofet, A. Gonzalvo, E. Alarcón, C. G. Almudéver, and S. Abadal, Route-forcing: Scalable quantum circuit mapping for scalable quantum computing architectures, in *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Vol. 1 (IEEE, 2024) pp. 909–920.
 - [21] D. Loss and D. P. DiVincenzo, Quantum computation with quantum dots, *Physical Review A* **57**, 120 (1998).
 - [22] A. Wild, J. Kierig, J. Sailer, J. W. Ager, E. Haller, G. Abstreiter, S. Ludwig, and D. Bougeard, Few electron double quantum dot in an isotopically purified 28Si quantum well, *Applied Physics Letters* **100** (2012).
 - [23] L. M. Vandersypen and M. A. Eriksson, Quantum computing with semiconductor spins, *Physics Today* **72**, 38 (2019).
 - [24] M. Veldhorst, J. Hwang, C. Yang, A. Leenstra, B. de Ronde, J. Dehollain, J. Muhonen, F. Hudson, K. M. Itoh, A. t. Morello, *et al.*, An addressable quantum dot qubit with fault-tolerant control-fidelity, *Nature nanotechnology* **9**, 981 (2014).
 - [25] E. Kawakami, T. Jullien, P. Scarlino, D. R. Ward, D. E. Savage, M. G. Lagally, V. V. Dobrovitski, M. Friesen, S. N. Coppersmith, M. A. Eriksson, *et al.*, Gate fidelity and coherence of an electron spin in an si/sige quantum dot with micromagnet, *Proceedings of the National Academy of Sciences* **113**, 11738 (2016).
 - [26] J. Yoneda, K. Takeda, T. Otsuka, T. Nakajima, M. R. Delbecq, G. Allison, T. Honda, T. Kodera, S. Oda, Y. Hoshi, *et al.*, A quantum-dot spin qubit with coherence limited by charge noise and fidelity higher than 99.9%, *Nature nanotechnology* **13**, 102 (2018).
 - [27] X. Xue, M. Russ, N. Samkharadze, B. Undseth, A. Sammak, G. Scappucci, and L. M. Vandersypen, Quantum logic with spin qubits crossing the surface code threshold, *Nature* **601**, 343 (2022).
 - [28] F. A. Zwanenburg, A. S. Dzurak, A. Morello, M. Y. Simmons, L. C. Hollenberg, G. Klimeck, S. Rogge, S. N.

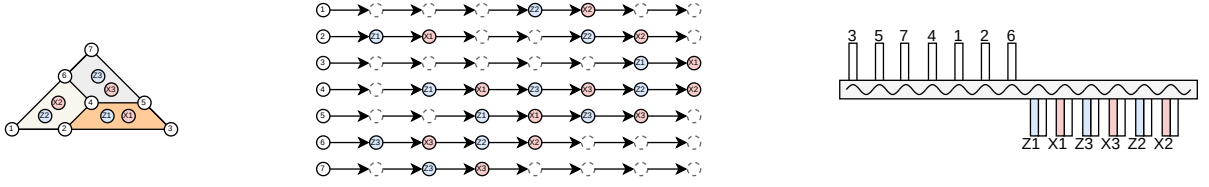


FIG. 21: Stabilizer layout, ancilla–data interaction structure, and optimal architecture synthesis for the 2D color code of distance $d = 3$. The six data-qubit interactions associated with each stabilizer define an acyclic dependency graph suitable for the MILP-based Quantum Reverse Mapping framework.

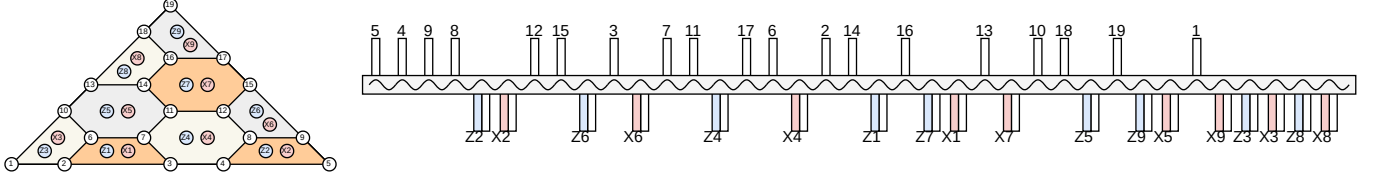


FIG. 22: Stabilizer layout and optimal architecture synthesis for the 2D color code of distance $d = 5$.

- Coppersmith, and M. A. Eriksson, Silicon quantum electronics, *Reviews of modern physics* **85**, 961 (2013).
- [29] A. Mills, D. Zajac, M. Gullans, F. Schupp, T. Hazard, and J. R. Petta, Shuttling a single charge across a one-dimensional array of silicon quantum dots, *Nature communications* **10**, 1063 (2019).
- [30] B. Jadot, P.-A. Mortemousque, E. Chanrion, V. Thiney, A. Ludwig, A. D. Wieck, M. Urdampilleta, C. Bäuerle, and T. Meunier, Distant spin entanglement via fast and coherent electron shuttling, *Nature Nanotechnology* **16**, 570 (2021).
- [31] I. Seidler, T. Struck, R. Xue, N. Focke, S. Trelenkamp, H. Bluhm, and L. R. Schreiber, Conveyor-mode single-electron shuttling in si/sige for a scalable quantum computing architecture, *npj Quantum information* **8**, 100 (2022).
- [32] A. Zwerver, S. Amitonov, S. de Snoo, M. Mądzik, M. Rimbach-Russ, A. Sammak, G. Scappucci, and L. Vandersypen, Shuttling an electron spin through a silicon quantum dot array, *PRX Quantum* **4**, 030303 (2023).
- [33] B. Buonacorsi, Z. Cai, E. B. Ramirez, K. S. Willick, S. M. Walker, J. Li, B. D. Shaw, X. Xu, S. C. Benjamin, and J. Baugh, Network architecture for a topological quantum computer in silicon, *Quantum Science and Technology* **4**, 025003 (2019).
- [34] J. M. Boter, J. P. Dehollain, J. P. Van Dijk, Y. Xu, T. Hensgens, R. Versluis, H. W. Naus, J. S. Clarke, M. Veldhorst, F. Sebastiano, *et al.*, Spiderweb array: a sparse spin-qubit array, *Physical review applied* **18**, 024053 (2022).
- [35] S. M. Patomäki, M. F. Gonzalez-Zalba, M. A. Fogarty, Z. Cai, S. C. Benjamin, and J. J. L. Morton, Pipeline quantum processor architecture for silicon spin qubits, *npj Quantum Information* **10**, 31 (2024).
- [36] M. Künne, A. Willmes, M. Oberländer, C. Gorjaew, J. D. Teske, H. Bhardwaj, M. Beer, E. Kammerloher, R. Otten, I. Seidler, *et al.*, The spinbus architecture for scaling spin qubits with electron shuttling, *Nature Communications* **15**, 4977 (2024).
- [37] J. M. Taylor, H. A. Engel, W. Dür, A. Yacoby, C. M. Marcus, P. Zoller, and M. D. Lukin, Fault-tolerant architecture for quantum computation using electrically controlled semiconductor spins, *Nature Physics* **1**, 177 (2005).
- [38] L. C. L. Hollenberg, A. D. Greentree, A. G. Fowler, and C. J. Wellard, Two-dimensional architectures for donor-based quantum computing, *Physical Review B* **74**, 045311 (2006).
- [39] Y. Matsumoto, M. D. Smet, L. Tryputen, S. L. de Snoo, S. V. Amitonov, A. Sammak, M. Rimbach-Russ, G. Scappucci, and L. M. K. Vandersypen, Two-qubit logic and teleportation with mobile spin qubits in silicon, *arXiv:2503.15434 [quant-ph]* (2025).
- [40] A. Németh, V. K. Bandaru, P. Alves, M. P. Losert, E. Brann, O. M. Eskandari, H. Soomro, A. Vivrekar, M. A. Eriksson, and M. Friesen, Omnidirectional shuttling to avoid valley excitations in si/sige quantum wells (2024), *arXiv:2412.09574 [cond-mat.mes-hall]*.
- [41] O. Crawford, J. Cruise, N. Mertig, and M. Gonzalez-Zalba, Compilation and scaling strategies for a silicon quantum processor with sparse two-dimensional connectivity, *npj Quantum Information* **9**, 13 (2023).
- [42] R. Xue, M. Beer, I. Seidler, S. Humpohl, J.-S. Tu, S. Trelenkamp, T. Struck, H. Bluhm, and L. R. Schreiber, Si/SiGe QuBus for single electron information-processing devices with memory and micron-scale connectivity function, *Nat Commun* **15**, 2296 (2024).
- [43] M. D. Smet, Y. Matsumoto, A.-M. J. Zwerver, L. Tryputen, S. L. de Snoo, S. V. Amitonov, A. Sammak, N. Samkharadze, Önder Gül, R. N. M. Wasserman, M. Rimbach-Russ, G. Scappucci, and L. M. K. Vandersypen, High-fidelity single-spin shuttling in silicon, *Nature Nanotechnology* **20**, 866 (2025).
- [44] M. Jeon, S. C. Benjamin, and A. J. Fisher, Robustness of electron charge shuttling: Architectures, pulses, charge defects, and noise thresholds, *Phys. Rev. B* **111**, 195302 (2025).
- [45] A. Sokolov, C. Power, and E. Blokhina, Physics-informed optimisation of conveyor mode spin qubit transport, *arXiv preprint arXiv:2510.06943* (2025).
- [46] R. Nagai, T. Takemoto, Y. Wachi, and H. Mizuno, Digitally controlled conveyor-belt spin shuttling in silicon for

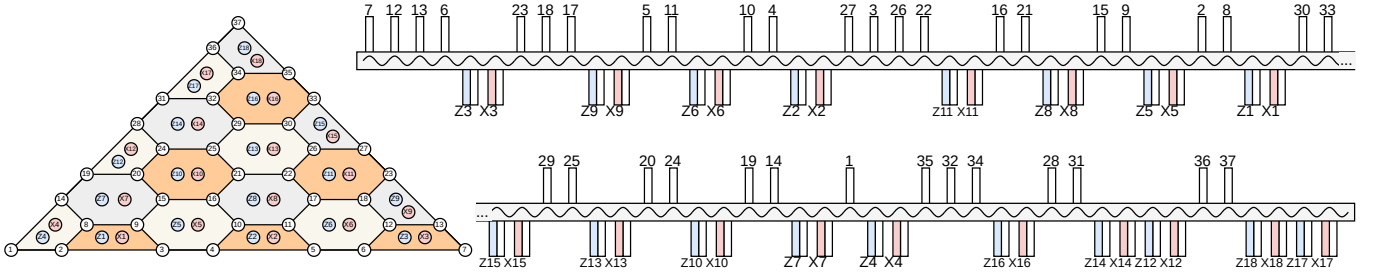


FIG. 23: Stabilizer layout and optimal architecture synthesis for the 2D color code of distance $d = 7$.

- large-scale quantum computation, *Phys. Rev. Appl.* **24**, 064021 (2025).
- [47] M. P. Losert, M. A. Eriksson, R. Joynt, R. Rahman, G. Scappucci, S. N. Coppersmith, and M. Friesen, Practical strategies for enhancing the valley splitting in si/sige quantum wells, *Phys. Rev. B* **108**, 125405 (2023).
- [48] V. Langrock, J. A. Krzywda, N. Focke, I. Seidler, L. R. Schreiber, and Ł. Cywiński, Blueprint of a scalable spin qubit shuttle device for coherent mid-range qubit transfer in disordered si/sige/sio₂, *PRX Quantum* **4**, 020305 (2023).
- [49] T. Fujita, T. A. Baart, C. Reichl, W. Wegscheider, and L. M. K. Vandersypen, Coherent shuttle of electron-spin states, *npj Quantum Information* **3**, 22 (2017).
- [50] H. Flentje, P.-A. Mortemousque, R. Thalneau, A. Ludwig, A. D. Wieck, C. Bäuerle, and T. Meunier, Coherent long-distance displacement of individual electron spins, *Nature Communications* **8**, 501 (2017).
- [51] D. D. Esposti, L. E. A. Stehouwer, Önder Gül, N. Samkharadze, C. D'eprez, M. Meyer, I. N. Meijer, L. Tryputen, S. Karwal, M. Botifoll, J. Arbiol, S. V. Amitonov, L. M. K. Vandersypen, A. Sammak, M. Veldhorst, and G. Scappucci, Low disorder and high valley splitting in silicon, *npj Quantum Information* **10**, 32 (2024).
- [52] D. A. Pucknell and K. Eshraghian, *Basic VLSI design* (Prentice-Hall, Inc., 1994).
- [53] C. Sechen and A. Sangiovanni-Vincentelli, Timberwolf3.2: A new standard cell placement and global routing package, in *23rd ACM/IEEE Design Automation Conference* (IEEE, 1986) pp. 432–439.
- [54] K. Yin, H. Zhang, X. Fang, Y. Shi, T. S. Humble, A. Li, and Y. Ding, Qecc-synth: A layout synthesizer for quantum error correction codes on sparse architectures, in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (2025) pp. 876–890.
- [55] G. Li, Y. Ding, and Y. Xie, Towards efficient superconducting quantum processor architecture design, in *Proceedings of the twenty-fifth international conference on architectural support for programming languages and operating systems* (2020) pp. 1031–1045.
- [56] T. Tomesh and M. Martonosi, Quantum codesign, *IEEE Micro* **41**, 33 (2021).
- [57] G. Li, A. Wu, Y. Shi, A. Javadi-Abhari, Y. Ding, and Y. Xie, On the co-design of quantum software and hardware, in *Proceedings of the Eight Annual ACM International Conference on Nanoscale Computing and Communication* (2021) pp. 1–7.
- [58] W.-H. Lin, B. Tan, M. Y. Niu, J. Kimko, and J. Cong, Domain-specific quantum architecture optimization, *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* **12**, 624 (2022).
- [59] L. Vandersypen, H. Bluhm, J. Clarke, A. Dzurak, R. Ishihara, A. Morello, D. Reilly, L. Schreiber, and M. Veldhorst, Interfacing spin qubits in quantum dots and donors—hot, dense, and coherent, *npj Quantum Information* **3**, 34 (2017).
- [60] R. Li, L. Petit, D. P. Franke, J. P. Dehollain, J. Helsen, M. Steudtner, N. K. Thomas, Z. R. Yoscovits, K. J. Singh, S. Wehner, *et al.*, A crossbar network for silicon quantum dot qubits, *Science advances* **4**, eaar3960 (2018).
- [61] F. Mohiyaddin, R. Li, S. Brebels, G. Simion, N. I. Dumoulin Stuyck, C. Godfrin, M. Shehata, A. Elsayed, B. Gys, S. Kubicek, J. Jussot, Y. Canvel, S. Massar, P. Weckx, P. Matagne, M. Mongillo, B. Govoreanu, and I. P. Radu, Large-scale 2d spin-based quantum processor with a bi-linear architecture, in *2021 IEEE International Electron Devices Meeting (IEDM)* (2021) pp. 27.5.1–27.5.4.
- [62] Z. Cai, A. Siegel, and S. Benjamin, Looped pipelines enabling effective 3d qubit lattices in a strictly 2d device, *PRX Quantum* **4**, 020345 (2023).
- [63] B. Yenilen, A. Sala, H. Bluhm, M. Müller, and M. Risper, Performance of the spin qubit shuttling architecture for a surface code implementation (2025), arXiv:2503.10601 [quant-ph].
- [64] P. Escofet, A. Semenov, N. Murphy, E. Blokhina, S. Abadal, E. Alarcón, and C. G. Almudéver, Compilation techniques for spin qubits in a shuttling bus architecture, in *2025 IEEE International Symposium on Circuits and Systems (ISCAS)* (2025).
- [65] N. Paraskevopoulos, C. G. Almudever, and S. Feld, besnake: A routing algorithm for scalable spin-qubit architectures, *IEEE Transactions on Quantum Engineering* (2024).
- [66] Y. Zhao, Y. Ye, H.-L. Huang, Y. Zhang, D. Wu, H. Guan, Q. Zhu, Z. Wei, T. He, S. Cao, *et al.*, Realization of an error-correcting surface code with superconducting qubits, *Physical Review Letters* **129**, 030501 (2022).
- [67] R. Barends, J. Kelly, A. Megrant, A. Veitia, D. Sank, E. Jeffrey, T. C. White, J. Mutus, A. G. Fowler, B. Campbell, *et al.*, Superconducting quantum circuits at the surface code threshold for fault tolerance, *Nature* **508**, 500 (2014).
- [68] C. J. Trout, M. Li, M. Gutiérrez, Y. Wu, S.-T. Wang, L. Duan, and K. R. Brown, Simulating the performance of a distance-3 surface code in a linear ion trap, *New Journal of Physics* **20**, 043038 (2018).

- [69] M. Kang, W. C. Campbell, and K. R. Brown, Quantum error correction with metastable states of trapped ions using erasure conversion, *PRX Quantum* **4**, 020358 (2023).
- [70] A. R. O’Rourke and S. Devitt, Compare the pair: Rotated vs. unrotated surface codes at equal logical error rates (2024), arXiv:2409.14765 [quant-ph].
- [71] Y. Tomita and K. M. Svore, Low-distance surface codes under realistic quantum noise, *Physical Review A* **90**, 062320 (2014).
- [72] C. Gidney, Stim: a fast stabilizer circuit simulator, *Quantum* **5**, 497 (2021).
- [73] K. Thulasiraman and M. Swamy, 5.7 acyclic directed graphs, *Graphs: theory and algorithms* **118**, 19 (1992).
- [74] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithms* (MIT press, 2022).
- [75] A. B. Kahn, Topological sorting of large networks, *Communications of the ACM* **5**, 558 (1962).
- [76] D. Bertsimas and J. N. Tsitsiklis, *Introduction to linear optimization*, Vol. 6 (Athena scientific Belmont, MA, 1997).
- [77] A. Schrijver, *Theory of linear and integer programming* (John Wiley & Sons, 1998).
- [78] Gurobi Optimization, LLC, Gurobi Optimizer Reference Manual (2024).
- [79] G. B. Dantzig, Programming in a linear structure, in *Bulletin of the American Mathematical Society*, Vol. 54 (1948) pp. 1074–1074.
- [80] G. B. Dantzig, Programming of interdependent activities: Ii mathematical model, *Econometrica, Journal of the Econometric Society*, 200 (1949).
- [81] C. E. Lemke, The dual method of solving the linear programming problem, *Naval Research Logistics Quarterly* **1**, 36 (1954).
- [82] A. H. Land and A. G. Doig, An automatic method for solving discrete programming problems, in *50 Years of Integer Programming 1958-2008: From the Early Years to the State-of-the-Art* (Springer, 2009) pp. 105–132.
- [83] I. I. Dikin, Iterative solution of problems of linear and quadratic programming, in *Soviet Math. Dokl.*, Vol. 8 (1967) pp. 674–675.
- [84] N. Karmarkar, A new polynomial-time algorithm for linear programming, in *Proceedings of the sixteenth annual ACM symposium on Theory of computing* (1984) pp. 302–311.
- [85] M. P. Losert, M. Oberländer, J. D. Teske, M. Volmer, L. R. Schreiber, H. Bluhm, S. Coppersmith, and M. Friesen, Strategies for enhancing spin-shuttling fidelities in si/si ge quantum wells with random-alloy disorder, *PRX Quantum* **5**, 040322 (2024).
- [86] H. Bombin and M. A. Martin-Delgado, Topological quantum distillation, *Physical review letters* **97**, 180501 (2006).
- [87] A. J. Landahl, J. T. Anderson, and P. R. Rice, Fault-tolerant quantum computing with color codes, arXiv preprint arXiv:1108.5738 (2011).
- [88] C. H. Papadimitriou, On the complexity of integer programming, *Journal of the ACM (JACM)* **28**, 765 (1981).
- [89] O. Higgott, Pymatching: A python package for decoding quantum codes with minimum-weight perfect matching, *ACM Transactions on Quantum Computing* **3** (2022).
- [90] F. Ginzl, M. Fellner, C. Ertler, L. R. Schreiber, H. Bluhm, and W. Lechner, Scalable parity architecture with a shuttling-based spin qubit processor, *Physical Review B* **110**, 075302 (2024).
- [91] D. J. Ibberson, T. Lundberg, J. A. Haigh, L. Hutin, B. Bertrand, S. Barraud, C.-M. Lee, N. A. Stelmashenko, G. A. Oakes, L. Cochrane, *et al.*, Large dispersive interaction between a cmos double quantum dot and microwave photons, *PRX Quantum* **2**, 020315 (2021).
- [92] A. P. Higginbotham, F. Kuemmeth, M. P. Hanson, A. C. Gossard, and C. M. Marcus, Coherent operations and screening in multielectron spin qubits, *Physical Review Letters* **112**, 026801 (2014).
- [93] C. Barthel, M. Kjærgaard, J. Medford, M. Stopa, C. M. Marcus, M. Hanson, and A. C. Gossard, Fast sensing of double-dot charge arrangement and spin state with a radio-frequency sensor quantum dot, *Physical Review B* **81**, 161308 (2010).
- [94] P. Stano and D. Loss, Review of performance metrics of spin qubits in gated semiconducting nanostructures, *Nature Reviews Physics* **4**, 672 (2022).
- [95] T. Struck, M. Volmer, L. Visser, T. Offermann, R. Xue, J.-S. Tu, S. Trellenkamp, Ł. Cywiński, H. Bluhm, and L. R. Schreiber, Spin-epr-pair separation by conveyor-mode single electron shuttling in si/sige, *Nature Communications* **15**, 1325 (2024).
- [96] K. W. Chan, H. Sahasrabudhe, W. Huang, Y. Wang, H. C. Yang, M. Veldhorst, J. C. C. Hwang, F. A. Mohiyaddin, F. E. Hudson, K. M. Itoh, A. Saraiva, A. Morello, A. Laucht, R. Rahman, and A. S. Dzurak, Exchange coupling in a linear chain of three quantum-dot spin qubits in silicon, *Nano Letters* **21**, 1517 (2021), pMID: 33481612.
- [97] P. Escofet, Quantum reverse mapping: Synthesizing an optimal spin qubit shuttling bus architecture for the surface code (2025).
- [98] S.-H. Lee, A. Li, and S. D. Bartlett, Color code decoder with improved scaling for correcting circuit-level noise, *Quantum* **9**, 1609 (2025).